UE App Development 623.9[51|52] SS 2026
Institute for Information Technology (ITEC) Primus|Münzer

ASSIGNMENT 01
Due date: 2026/03/[19|17]

Aim: Familiarization with Kotlin and tools for Android app development.

1 Simple Kotlin Example

Implement the following scenario using idiomatic Kotlin features like **properties**, **data classes**, and **null safety**.

Data Model

Implement the following classes as **data classes**. Use **default values** to allow flexible constructor calls.

- **Brand**:
  - `name: String`, `country: String` (Default: `"Austria"`), `phone: String`, `email: String?` (Nullable, Default: `null`).
- **Workshop**:
  - `name, country, city, street, phone` (all `String`).
  - `postcode`: Internally stored as a `String`. Implement the class so that it accepts both **Int** and **String** during initialization (using a secondary constructor). Ensure the postcode is always stored as a String internally, regardless of which constructor is used.

The Vehicle Class

Implement the *Vehicle* class with a primary constructor. Ensure proper use of `val` for immutable and `var` for mutable properties.

Property	Type	Note / Purpose
`id`	`Int`	**read-only** (`val`)
`name`	`String`	mutable name of the vehicle
`brand`	`Brand`	the manufacturer
`workshops`	`MutableList<Workshop>`	Default: `mutableListOf()`
`weight`	`Int`	**current** weight in kg
`maxWeight`	`Int`	maximum permitted weight
`speed`	`Double`	**current** speed (Default: `0.0`)
`maxSpeed`	`Double`	maximum possible speed (Default: `200.0`)

Computed Properties & Custom Getters

Kotlin allows properties to have custom logic. Implement the following **computed property** inside the class body:

- `isOverloaded: Boolean`: This property should not be a stored field. Instead, use a **custom getter** (`get()`) that dynamically calculates if `weight > maxWeight`.
- **Integration:** Update your `printInfo()` method to include a warning message (e.g., *"CAUTION: Overloaded!"*) using this property.

Methods and Control Flow

Implement the following functions. Avoid classic `for`-loops and use Kotlin's **Ranges** and **Standard Functions** (like `repeat`).

Function	Behavior / Purpose
`fun accelerate(): Double`	Increases `speed` by a random value in the range `10.0..50.0`, up to `maxSpeed`. Returns the new speed.
`fun brake(): Double`	Reduces `speed` by a random value in the range `10.0..50.0`, down to `0.0`. Returns the new speed.
`fun drive(km: Int)`	Simulates a trip. For each kilometer, execute a random number of actions (`3..5`). Randomly decide whether to accelerate or brake and **immediately print the returned speed** of each action to the console (using String Templates).
`fun printInfo()`	Prints all details using **String Templates**. Handle the nullable `email` using the **Elvis Operator** (`?:`) to show `"N/A"` if null.
`fun findWorkshop(postcode: String)`	Returns a `Workshop?` (nullable) with the matching postcode.

Kotlin Idiom: Ranges & Random Decisions

To implement the random behavior effectively:

- **Random Choice:** Use `Random.nextBoolean()` to decide between two actions:

```
if (Random.nextBoolean()) accelerate() else brake()
```

- **Iteration:** Nest a `repeat()` block for the actions inside a `repeat(km)` block for the distance.

Testing your Implementation

To verify your implementation, create a separate file (e.g., `Main.kt`). In Kotlin, the entry point is a simple top-level function; no class wrapper is required.

- **Entry Point:** Implement a `fun main()`.

- **Object Creation:**
  - Create at least one `Brand` and a few `Workshop` objects.
  - Instantiate multiple `Vehicle` objects. Use **Named Arguments** to make your code readable:
    `val car = Vehicle(id = 1, name = "Golf", maxWeight = 1500)`
  - Create at least one vehicle where the `weight` exceeds the `maxWeight` to test your **custom getter** logic.
- **Simulation Flow:**
  1. Add the workshops to your vehicles' `workshops` list.
  2. Store all your vehicles in a `List` (e.g., using `listOf(...)`).
  3. Iterate through the list and for each vehicle:
     - Call `printInfo()` to see the initial state.
     - Simulate a trip of 10 kilometers using `drive(10)`.
     - Call `printInfo()` again to see the updated speed and weight/overload warnings.
- **Output Formatting:** Ensure that your `printInfo()` implementation uses **String Templates** (`"$variable"`). For the overloaded warning, use your `isOverloaded` property:
  `if (isOverloaded) println("!!!  WARNING: Overloaded !!!")`

Hint: Random Numbers

Import `kotlin.random.Random`. You can use `Random.nextDouble(10.0, 50.0)` for speeds or the range syntax `(3..5).random()` for integer counts.

Recommended Resources

Development Environment: We recommend using **IntelliJ IDEA** (Community or Ultimate) or **Android Studio**. Both provide excellent support for Kotlin's static analysis and refactoring tools.

- `https://www.jetbrains.com/idea/`

Learning Kotlin: Familiarize yourself with idiomatic Kotlin (Properties, Null Safety, Lambdas) using the official documentation and interactive exercises:

- **Kotlin Docs:** `https://kotlinlang.org/docs/home.html`
- **Kotlin Koans:** Practical exercises to learn the syntax (`https://kotlinlang.org/docs/koans.html`)
- **Android Kotlin Basics:** `https://developer.android.com/courses/android-basics-kotlin/course`

2 Kotlin Coroutines

Kotlin coroutines are a very useful approach for asynchronous programming and concurrency management. They facilitate non-blocking execution of tasks such as network requests and UI interactions, replacing cumbersome callback structures and traditional threading mechanisms. With their lightweight design, coroutines are indispensable for creating responsive applications across platforms.

- ☐ Make yourself familiar with Kotlin Coroutines (`https://kotlinlang.org/docs/coroutines-overview.html`)
- ☐ Explain the difference between a Coroutine and a Thread
- ☐ What is the meaning of the *suspend* keyword?
- ☐ Extend your implementation of Exercise 1 with Coroutines to simulate a simple race between different Vehicles (hint: you might want to use the *delay()* function between random *accelerate()* and *brake()* calls.)

3 Lambda- and Scope Functions

Develop a small program that processes a list of contacts. In this task, you will use lambda functions for filtering and processing data, as well as scope functions for initializing and handling objects.

- ☐ Define a data class `Contact` with the following properties:
  - `name: String`
  - `email: String?`
  - `phone: String?`
  - `age: Int?`
  - (Optional) Additional properties such as address, ...
- ☐ Initialize Data
  - Create a list of at least 5 contacts.
  - Use scope functions such as `apply` or `run` to configure the contacts immediately after their creation.
- ☐ Process the List: Utilize lambda functions (such as `filter`, `map`, ...) to process the list of contacts
  - **Filtering:** Create a function that filters all contacts with a valid email address.
  - **Mapping:** Generate a new list (e.g., a list of strings containing only the name and email) from the existing contacts list.
  - **Accumulation:** Calculate the average age of the contacts.

- ☐ Apply Scope Functions: Employ scope functions like `let`, `also`, `run`, or `with` to work with the objects in different contexts:
  - Use `let` to process a contact only if its email address is not `null`.
  - Use `apply` to set properties of a contact within a configuration block.
  - Use `with` to perform multiple operations on a contact without repeatedly referencing the object.
  - Implement a search function (e.g., `findByName`) that uses a lambda function to search for a contact by name.
- ☐ Documentation and Comments:
  - Comment your code adequately to explain the usage of lambda functions and scope functions.
  - Describe each scope function (name, context(receiver), return value, typical application, ...) in a tabular overview.

4 IDE and Build System

4.1 Android Studio

Android Studio is an integrated development environment (IDE) for the development of applications for Google's Android OS. It is based on IntelliJ IDEA software from JetBrains. Android Studio supports several programming languages including Kotlin since version 3.0. In this task, you will make yourself familiar with this powerful tool that we will use in this course.

- ☐ Install Android Studio in your OS
  Have a look at `https://developer.android.com/studio/install`
- ☐ Create sample projects with `File > New > New Project` and inspect:
  - Resource Manager Window
  - Project Window
  - Structure Window
- ☐ Install and launch the Android Emulator
  - ☐ Optional: Enable hardware acceleration
  - ☐ Create and manage virtual devices with different properties
  - ☐ Explore how to perform basic control actions with preinstalled apps
  - ☐ Explore how to use extended controls (location, displays, sensors, ...) with preinstalled apps
- ☐ Explore Logcat

- ☐ Get familiar with the Logcat window in Android Studio
- ☐ Explain the purpose of logcat and how messages can be filtered

☐ Inspect the Device File Explorer Window in Android Studio

4.2 Gradle

Gradle is used as build system in Android Studio. Android studio automatically generates gradle build files. These build files might be sufficient for small projects but need to be manually edited for release builds or more complex software.

☐ Get a closer look at gradle and describe the terms, e.g., build type, product flavor, build variants, dependencies.

☐ Describe the build process that converts a project into an Android Application Package (APK).

☐ Describe following files used in gradle:
`gradle.properties, settings.gradle[.kts], build.gradle[.kts]`